

LAB ASSIGNMENT - 7.2

NAME – Anushka Boora

Ht No.: 2303A510E6

BATCH – 29

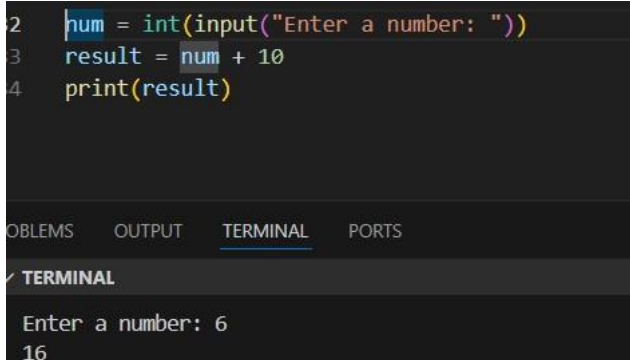
Task 1 – Runtime Error Due to Invalid Input Type

PROMPT :

Getting a run time error identify the cause for the error and fix it.

CODE :

```
2  num = int(input("Enter a number: "))
3  result = num + 10
4  print(result)
```



Justification :

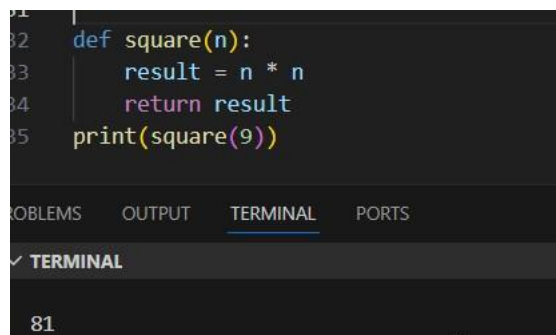
AI helped me understand that the input() function in Python returns a string by default, which caused a TypeError when performing arithmetic operations. By converting the input into an integer using type casting, the runtime error was resolved successfully. This task improved my understanding of data types and their importance in arithmetic operations. It also increased my confidence in identifying and fixing type-related runtime errors.

Task 2 Incorrect Function Return Value PROMPT :

Not getting an output identify the cause for the error and fix it.

CODE :

```
1  def square(n):
2      result = n * n
3      return result
4  print(square(9))
```



Justification :

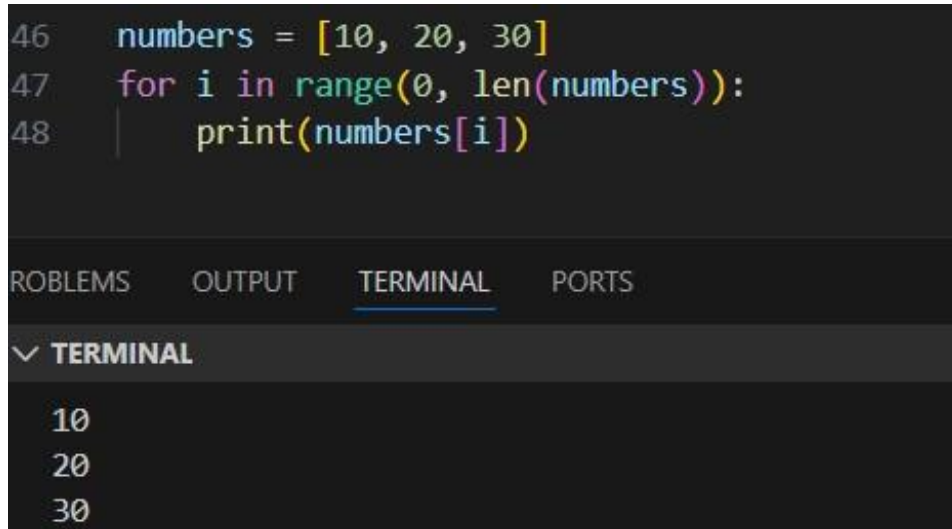
AI assisted in identifying that the function was missing a return statement, which caused it to return None instead of the expected value. By adding the appropriate return statement, the function was able to correctly provide the computed result. This helped me better understand how functions work internally and why returning values is important. It strengthened my knowledge of function structure and behavior in Python. **Task 3**

IndexError in List Traversal PROMPT :

Getting an index out of range error identify the cause for the error and fix it.

CODE :

```
46 numbers = [10, 20, 30]
47 for i in range(0, len(numbers)):
48     print(numbers[i])
```

The image shows a screenshot of a code editor with a dark theme. The code is written in Python and consists of three lines: line 46 defines a list 'numbers' with values [10, 20, 30]; line 47 starts a for loop with 'i' ranging from 0 to the length of 'numbers' (exclusive); line 48 prints the element at index 'i' of the 'numbers' list. Below the code, there are tabs for 'PROBLEMS', 'OUTPUT', 'TERMINAL', and 'PORTS'. The 'TERMINAL' tab is selected and shows the output of the program: '10', '20', and '30' on separate lines.

Justification :

AI detected that the loop boundary condition exceeded the valid index range of the list, which caused an IndexError. By correcting the loop range and removing the extra increment, the program executed without errors. This task helped me understand the importance of proper indexing and boundary conditions when working with lists. It also improved my debugging skills related to iteration logic.

Task 4 Uninitialized Variable Usage PROMPT

:

Getting a name error identify the cause for the error and fix it.

CODE :

```
50 if True:
51     total = 0
52     print(total)
53
54
```

PROBLEMS OUTPUT TERMINAL PORTS

▼ **TERMINAL**

0

Justification :

AI identified that a variable was being used before it was initialized, leading to a NameError. By initializing the variable before using it in the program, the error was successfully resolved. This task highlighted the importance of defining variables before usage. It enhanced my understanding of variable scope and initialization in Python.

Task 5 Logical Error in Student Grading System **PROMPT**

:

Getting wrong output identify the cause for the error and fix it.

CODE :

```
marks = 85
if marks >= 90:
    grade = "A"
elif marks >= 80:
    grade = "C"
else:
    grade = "B"
print(grade)
```

PROBLEMS OUTPUT TERMINAL PORTS

TERMINAL

C

Justification :

AI analyzed the grading logic and found that the conditional structure was incorrectly assigning grades. By reorganizing and correcting the conditional statements, the program was able to assign accurate grades based on marks. This task improved my understanding of

logical flow and condition hierarchy in decision-making statements. It also helped me realize how logical errors can affect program correctness even without syntax issues.